

Seamless Transition Between Gaussian Splatting and Mesh Rendering

Jerneja Krajcar

University of Ljubljana

Abstract

This seminar presents a prototype for distance-dependent transitions between mesh rendering and 3D Gaussian Splatting. The goal is to use a mesh representation for distant, low-detail views and a Gaussian representation for close-up views, while avoiding abrupt visual changes. The implemented solution builds Gaussian levels of detail, computes smooth transition weights from camera distance, blends mesh and Gaussian renderings, and exposes the same behaviour in an offline pipeline and a web viewer.

Keywords: 3D Gaussian Splatting, mesh rendering, level of detail, hybrid rendering, smooth transitions

1. Introduction

Meshes are compact, widely supported, and reliable for low-detail views. 3D Gaussian Splatting (3DGS), introduced by Kerbl et al. [KKLD23], is useful for high-quality appearance because it represents a scene with many semi-transparent Gaussian primitives instead of only triangle surfaces. The seminar task is therefore a hybrid rendering problem: use the scalable behaviour of meshes at distance, but switch to richer Gaussian detail when the camera moves closer.

The main difficulty is not only selecting the right representation, but changing between representations without popping, brightness jumps, or visible discontinuities. The current implementation is a prototype, not a production renderer. It is designed so that the transition logic, data flow, and evaluation outputs remain easy to inspect and extend.

2. Problem

The required system should support a seamless transition between two scene representations:

- a simplified mesh, suitable for low-detail or far-away views;
- one or more Gaussian splat representations, suitable for higher-detail close-up views.

A naive switch at a fixed distance produces visible popping because the two renderers do not necessarily match in color, coverage, silhouette, or sampling density. A better solution must make the representation change gradual and should keep neighbouring Gaussian levels spatially related, so that a denser level of detail does not look like a completely unrelated point set.

3. Related Work

3DGS provides the basic representation used for the Gaussian side of this project [KKLD23]. SuGaR aligns Gaussians with surfaces and uses this relationship for mesh extraction and high-quality mesh rendering [GL24]. This is directly relevant because it shows that mesh and Gaussian representations can be connected through surface alignment rather than treated as unrelated assets.

SplattingAvatar embeds Gaussian splats on a mesh for real-time human avatar rendering [SWL*24]. Its main target is animatable humans, but the idea of attaching or aligning Gaussian appearance to mesh structure influenced the planned direction of this seminar work. Mesh2Splat [Ele25] was also considered and integrated as an external conversion path because it offers a practical way to create Gaussian splat files from mesh assets.

4. Implemented Solution

The current prototype contains two complementary workflows.

First, the offline Python pipeline loads a mesh or creates a procedural demo sphere, normalizes it to a unit sphere, generates orbit cameras, samples the mesh surface, and builds nested Gaussian levels of detail. The LODs are nested by using a shared farthest-point ordering: smaller LODs are prefixes of the same ordered set. This keeps coarse and dense Gaussian levels spatially related and reduces transition instability.

Second, the web viewer supports interactive preparation and inspection. It can use an initialized preview built from mesh sampling, a single trained Gaussian PLY file, or a Mesh2Splat LOD set exported as several PLY files. This is useful because the simple initialized preview is easy to test, while the Mesh2Splat and trained-splat modes are closer to the intended final representation.

4.1. Transition Model

The transition is controlled by camera distance. A smoothstep function is used for all fades, so weights change gradually instead of linearly snapping at a threshold. The mesh weight fades out between configurable far and near distances. The remaining weight budget is assigned to active Gaussian LOD bands and then normalized, so that the combined mesh and Gaussian contribution stays stable.

For the default configuration, the mesh starts fading near distance 3.6 and is mostly replaced by Gaussian rendering near distance 2.2 in normalized object units. Gaussian LOD bands then progressively select counts such as 10, 100, 500, 5000, and 20000 splats as the camera moves closer.

4.2. Blending

The current blend is an image-space weighted sum:

$$I = w_m I_m + \sum_i w_i I_i,$$

where I_m is the mesh render, I_i are Gaussian LOD renders, and the weights sum to one. This keeps the transition simple and inspectable. It does not yet solve deeper alignment problems such as exact silhouette matching, depth-aware compositing between representations, or learned color correction.

5. Alternatives Considered

Hard LOD switching would be simpler, but it directly causes popping and therefore does not satisfy the main goal of the seminar task.

Depth-aware screen-space blending could reduce artifacts where the mesh and Gaussians overlap incorrectly. It is a good future direction, but the first version uses color blending because it is easier to debug and works with the current lightweight renderers.

Full surface-aligned Gaussian optimization in the style of SuGaR would likely improve geometric consistency, but it is heavier than the current seminar prototype and would require a stronger training and reconstruction pipeline.

Only using Mesh2Splat was also considered. The project integrates Mesh2Splat as an external path, but it is not the only path because the seminar implementation should remain runnable and testable without depending on a specific external Windows build.

6. Results

The prototype currently produces a transition video, saved Gaussian LOD files, and a metrics file with render timing, LOD memory use, quality metrics, transition weights, and a simple popping score.

On the current default demo sphere output, the generated LODs contain 10, 100, 500, 5000, and 20000 Gaussians. The stored Gaussian data scales linearly from 320 bytes for 10 Gaussians to 640000 bytes for 20000 Gaussians. The transition output contains

32 frames and logs the exact mesh and Gaussian LOD weights for every frame. This confirms that the implementation performs a continuous weighted transition instead of a single hard switch.

The CPU software renderer is intentionally simple and inspectable, so high Gaussian counts are slow. This is acceptable for validating the algorithmic structure, but real-time performance would require a GPU Gaussian rasterizer such as gsplat or another optimized backend.

7. Limitations and Future Work

The largest current limitation is that the initialized Gaussian preview is surface-sampled rather than fully optimized for photorealistic appearance. The project already includes paths for trained Gaussian PLY files and Mesh2Splat LOD sets, which are more appropriate for higher-quality demonstrations.

Future work should focus on three improvements: depth-aware blending, better alignment between mesh and Gaussian geometry, and quantitative comparison on real assets rather than only on the procedural demo. A later version could also bind Gaussian parameters more directly to mesh surfaces, following the direction suggested by SuGaR and SplattingAvatar.

8. Conclusion

The implemented prototype demonstrates a clear first step toward seamless mesh-to-Gaussian LOD transitions. It does not claim to solve all visual artifacts, but it establishes the key structure: multiple representations, distance-based smooth weights, nested Gaussian LODs, offline and interactive viewing, and metrics for inspecting the result. This direction was chosen because it is simple enough to validate now while still leaving a direct path toward more advanced alignment and blending methods.

References

- [Ele25] ELECTRONIC ARTS: Mesh2Splat: Fast mesh to 3d gaussian splat conversion. <https://github.com/electronicarts/mesh2splat>, 2025. 1
- [GL24] GUÉDON A., LEPETIT V.: SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/html/Guedon_SuGaR_Surface-Aligned-Gaussian-Splatting_for_Efficient_3D_Mesh_Reconstruction_and_CVPR_2024_paper.html. 1
- [KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (2023). doi:10.1145/3592433. 1
- [SWL*24] SHAO Z., WANG Z., LI Z., WANG D., LIN X., ZHANG Y., FAN M., WANG Z.: Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/papers/Shao_SplattingAvatar_Realistic_Real-Time_Human_Avatars_with_Mesh-Embedded_Gaussian_Splatting_CVPR_2024_paper.pdf. 1